МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

КУРСОВАЯ РАБОТА

по дисциплине «Программирование»

Тема: Разработка программы на языке C++ для обработки изображений

Студент гр. 3344 _____ Мурдасов М.К.

Преподаватель _____ Глазунов С.А.

Санкт-Петербург

2024

ЗАДАНИЕ
НА КУРСОВУЮ РАБОТУ

Студент: Мурдасов М.К.

Группа: 3344

Тема работы: Разработка программы на языке C++ для обработки изображений

Исходные данные:

- Написание программы должно производиться на языке C/C++ (на выбор)
- Реализация ***CLI*** должна быть с использованием ***getopt***.
- Ввод данных должен быть произведен в стандартный поток ввода: ***stdin/cin***.
- Вывод программы должен быть произведен в стандартный поток вывода: ***stdout/cout***.
- Вывод изображения должен быть произведен в новый файл формата .bmp (по умолчанию out.bmp)
- Должен быть написан *Makefile*. (Название исполняемого файла: ***cw***)

Содержание пояснительной записки:

- Содержание
- Введение
- Описание архитектуры программы
- Описание структур данных
- Описание реализованных функций
- Тестирование
- Примеры обработки ошибок
- Инструкция по запуску

- Заключение
- Список использованных источников

Предполагаемый объем пояснительной записки:

Не менее 50 страниц

Дата выдачи задания: 18.03.2024

Дата сдачи реферата: 23.05.2024

Дата защиты реферата: 23.05.2024

Студент гр. 3344	_____	Мурдасов М.К.
Преподаватель	_____	Глазунов С.А.

АННОТАЦИЯ

Курсовая работа написана на языке C++ и представляет собой программу для обработки изображений. Программа включает в себя функции чтения и обработки BMP файлов, а также функции обработки аргументов командной строки и ошибок. Методы исследования включают в себя работу со структурой BMP файла. Результаты работы включают в себя реализацию CLI, рисование квадрата, рисование окружности, фильтр RGB-компонент, поворот области изображения.

СОДЕРЖАНИЕ

ЗАДАНИЕ 2
АННОТАЦИЯ 4
СОДЕРЖАНИЕ 5
ВВЕДЕНИЕ 7
1. ЗАДАНИЕ 8
2. ОПИСАНИЕ АРХИТЕКТУРЫ ПРОГРАММЫ 11
2.1. Файловая структура проекта 11
2.2. Принцип работы 11
3. ОПИСАНИЕ СТРУКТУР ДАННЫХ 13
3.1 Структура файла BMP 13
3.1 Bitmap File Header 13
3.2 Bitmap Info Header 14
4. ОПИСАНИЕ РЕАЛИЗОВАННЫХ ФУНКЦИЙ 15
4.1 main.cpp 15
4.2 bmp_structs.hpp 15
4.3 drawCircle.cpp 15
4.4 drawSquare.cpp 15
4.5 rgbFilter.cpp 15
4.6 rotate.cpp 15
4.7 flagsProcessing.cpp 16
4.8 validateArgs.cpp 16
4.9 wrapper.cpp 17
4.10 readBmp.cpp 17
4.11 writeBmp.cpp 17
4.12 printInfo.cpp 18
5. ПОЛУЧЕННЫЕ РЕЗУЛЬТАТЫ 19
6. ТЕСТИРОВАНИЕ 20
7. ПРИМЕРЫ ОБРАБОТКИ ОШИБОК 29
ИНСТРУКЦИЯ ПО ЗАПУСКУ 33
ЗАКЛЮЧЕНИЕ 34

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 35
ПРИЛОЖЕНИЕ А 36

ВВЕДЕНИЕ

Цель работы: разработка программы на языке C++ для обработки изображений.

Основные задачи:

- Реализовать чтение/вывод файла bmp.
- Реализовать обработку аргументов с помощью *getopt*
- Реализовать проверку аргументов и их значений.
- Реализовать функции обработки изображений.
- Реализовать эффективную сборку программы.
- Реализовать обработку всевозможных ошибок и завершение работы программы с выводом подробной информации пользователю.

Методы решения:

- Реализация структуры *BMPImage* для удобной работы с bmp файлом.
- Использование *getopt* и *unordered_map* при разборе команд пользователя.
- Реализация функций-проверок к каждому типу аргументов.
- Разделение кода на модули для лучшей читаемости и гибкости проекта.
- Использование *Makefile* для сборки проекта.
- Создание информативной справки о программе для пользователя.

1. ЗАДАНИЕ

Программа **обязательно должна иметь CLI** (опционально дополнительное использование GUI). Более подробно тут:

http://se.moevm.info/doku.php/courses:programming:rules_extra_kurs

Программа должна реализовывать весь следующий функционал по обработке bmp-файла

Общие сведения

- 24 бита на цвет
- без сжатия
- файл может не соответствовать формату BMP, т.е. необходимо проверка на BMP формат (дополнительно стоит помнить, что версий у формата несколько). Если файл не соответствует формату BMP или его версии, то программа должна завершиться с соответствующей ошибкой.
- обратите внимание на выравнивание; мусорные данные, если их необходимо дописать в файл для выравнивания, должны быть нулями.
- обратите внимание на порядок записи пикселей
- все поля стандартных BMP заголовков в выходном файле должны иметь те же значения что и во входном (разумеется кроме тех, которые должны быть изменены).

Программа должна иметь следующую функции по обработке изображений:

1. Рисование квадрата с диагоналями. Флаг для выполнения данной операции: `--squared_lines`. Квадрат определяется:
    - Координатами левого верхнего угла. Флаг `--left_up`, значение задаётся в формате `left.up`, где left – координата по x, up – координата по y
    - Размером стороны. Флаг `--side_size`. На вход принимает число больше 0

- Толщиной линий. Флаг `--thickness`. На вход принимает число больше 0
- Цветом линий. Флаг `--color` (цвет задаётся строкой `rrr.ggg.bbb`, где rrr/ggg/bbb – числа, задающие цветовую компоненту. пример `--color 255.0.0` задаёт красный цвет)
- Может быть залит или нет (диагонали располагаются “поверх” заливки). Флаг `--fill`. Работает как бинарное значение: флага нет – false , флаг есть – true.
- Цветом которым он залит, если пользователем выбран залитый. Флаг `--fill_color` (работает аналогично флагу `--color`)

2. Фильтр rgb-компонент. Флаг для выполнения данной операции: `--rgbfilter`. Этот инструмент должен позволять для всего изображения либо установить в диапазоне от 0 до 255 значение заданной компоненты. Функционал определяется
   - Какую компоненту требуется изменить. Флаг `--component_name`. Возможные значения `red`, `green` и `blue`.
   - В какой значение ее требуется изменить. Флаг `--component_value`. Принимает значение в виде числа от 0 до 255
3. Поворот изображения (части) на 90/180/270 градусов. Флаг для выполнения данной операции: `--rotate`. Функционал определяется
   - Координатами левого верхнего угла области. Флаг `--left_up`, значение задаётся в формате `left.up`, где left – координата по x, up – координата по y
   - Координатами правого нижнего угла области. Флаг `--right_down`, значение задаётся в формате `right.down`, где right – координата по x, down – координата по y
   - Углом поворота. Флаг `--angle`, возможные значения: `90`, `180`, `270`
4. Рисование окружности. Флаг для выполнения данной операции: `--circle`. Окружность определяется:

- координатами ее центра и радиусом. Флаги `--center` и `--radius`. Значение флаг `--center` задаётся в формате `x.y`, где x – координата по оси x, y – координата по оси y. Флаг `--radius` На вход принимает число больше 0
- толщиной линии окружности. Флаг `--thickness`. На вход принимает число больше 0
- цветом линии окружности. Флаг `--color` (цвет задаётся строкой `rrr.ggg.bbb`, где rrr/ggg/bbb – числа, задающие цветовую компоненту. пример `--color 255.0.0` задаёт красный цвет)
- окружность может быть залитой или нет. Флаг `--fill`. Работает как бинарное значение: флага нет – false , флаг есть – true.
- цветом которым залита сама окружность, если пользователем выбрана залитая окружность. Флаг `--fill_color` (работает аналогично флагу `--color`)

Каждую подзадачу следует вынести в отдельную функцию, функции сгруппировать в несколько файлов (например, функции обработки текста в один, функции ввода/вывода в другой). Сборка должна осуществляться при помощи make и Makefile или другой системы сборки

2. ОПИСАНИЕ АРХИТЕКТУРЫ ПРОГРАММЫ

2.1. Файловая структура проекта

Программа состоит из нескольких модулей (файлов), отведенных под определенные задачи, что обеспечивает структурированность, гибкость и удобство работы с проектом.

Все функции разбиты на файлы, согласно своему назначению:

- *main.cpp* – главная функция проекта
- *bmp_structs.hpp* – определение структур bmp-файла
- *printInfo.cpp* – вывод информации (*--help* или информация об изображении)
- *drawCircle.cpp* – рисование окружности
- *drawSquare.cpp* – рисование квадрата с диагоналями
- *rgbFilter.cpp* – фильтр компонент цвета
- *rotate.cpp* – поворот области изображения
- *flagsProcessing.cpp* – обработка аргументов командной строки
- *validateArgs.cpp* – проверка значений аргументов
- *readBmp.cpp* – чтение bmp-файла
- *writeBmp.cpp* – запись bmp-файла
- *wrapper.cpp* – запуск всех процессов

Также каждому *.cpp* файлу, кроме main.cpp, соответствует свой заголовочный *.hpp* файл, содержащий сигнатуры используемых функций и необходимые библиотеки. Такой метод обеспечивает возможность использования этих функций в любых модулях программы.

2.2. Принцип работы

Работа программы организована следующим образом:

1. Аргументы командной строки передаются на разбор функции *getopt* и сохраняются в хеш-таблицу.
2. Из хеш-таблицы, среди всех аргументов выделяется имя опции.
3. Хеш-таблица аргументов отправляется на полную проверку, где проверяется корректность аргументов и значений этих аргументов.
4. Производится чтение bmp-файла по заданному пути.
5. Относительно введенных данных, производится разбор значений аргументов и их передача о определенную функцию.
6. Производится вывод обработанного изображения, если того требует операция.

Также на каждом этапе работы проекта производится обработка множества ошибок. В случае их возникновения пользователю выводится сообщение с подробной информацией об ошибке и подсказка о том, как вызвать справку.

3. ОПИСАНИЕ СТРУКТУР ДАННЫХ

3.1 Структура файла BMP

Блок	Размер	Обязательный параметр	Описание
Заголовок файла (Bitmap file header)	14 байт	Да	Используется для хранения общей информации об изображении
Заголовок DIB (Bitmap info header)	Фиксированный размер (7 версий)	Да	Используется для хранения подробной информации об изображении и определения формата пикселей
Дополнительные битовые маски (Extra bit masks)	12 или 16 байт	Нет	Используется для определения формата пикселей
Цветовая палитра (Color table)	Переменный размер	Полуопциональный*	Используется для определения цветов, используемых изображением (массивом пикселей)
Пробел1 (Gap1)	Переменный размер	Нет	Используется для выравнивания структуры
Массив пикселей (Pixel array)	Переменный размер	Да	Используется для определения фактических значений пикселей. Формат пикселей определяется заголовком DIB или дополнительными битовыми масками. Каждая строка в массиве заполняется до размера, кратного 4 байтам
Пробел2 (Gap2)	Переменный размер	Нет	Используется для выравнивания структуры
Цветовой профиль ICC (ICC color profile)	Переменный размер	Нет	Используется для определения цветового профиля для управления цветом

* Обязательно используется при битности цвета 8 и ниже, может использоваться при битности 8 и выше для оптимизации цветов.

3.1 Bitmap File Header

- *Signature* (2 байта) – заголовок, идентифицирующий файл bmp.
- *FileSize* (4 байта) – размер файла bmp в байтах
- *Reserved1* (2 байта) - зарезервировано
- *Reserved2* (2 байта) - зарезервировано

- *FileOffsetToPixelArray* (4 байта) – начальный адрес байта, в котором находится массив пикселей.

3.2 Bitmap Info Header

- *HeaderSize* (4 байта) – размер этого заголовка в байтах.
- *ImageWidth* (4 байта) – ширина изображения в пикселях.
- *ImageHeight* (4 байта) – длина изображения в пикселях.
- *Planes* (2 байта) – количество цветовых плоскостей.
- *BitsPerPixel* (2 байта) – глубина цвета изображения.
- *Compression* (4 байта) – используемый метод сжатия
- *ImageSize* (4 байта) – используемый метод сжатия.
- *XpixelsPerMeter* (4 байта) – горизонтальное разрешение изображения.
- *YpixelsPerMeter* (4 байта) – вертикальное разрешение изображения.
- *TotalColors* (4 байта) – количество цветов в цветовой палитре.
- *ImportantColors* (4 байта) – количество используемых важных цветов.

4. ОПИСАНИЕ РЕАЛИЗОВАННЫХ ФУНКЦИЙ

4.1 main.cpp

int main(); - основная функция программы. Содержит в себе вызовы функций получения аргументов, получения имени операции, полной проверки аргументов и их значений, чтения файла, запуск выбранной операции и вывод обработанного изображения. Также содержит проверку на отсутствие аргументов.

4.2 bmp_structs.hpp

Содержит объявление структур bmp-файла.

4.3 drawCircle.cpp

void fillCircle(); - рисует закрашенную окружность по алгоритму Брезенхема. Также используется для рисования линий с толщиной >1.

void drawCircle(); - рисует окружность, используя заданные параметры центра, радиуса, толщины линии, цвета линий. Опционально использование заливки определенного цвета.

4.4 drawSquare.cpp

void draw_square(); - рисует квадрат с диагоналями, используя заданные параметры левого верхнего угла, длины стороны, толщины линий, цвета линий. Опционально использование заливки определенного цвета.

4.5 rgbFilter.cpp

void rgbFilter(); - заменяет значение выбранной компоненты на заданное пользователем.

4.6 rotate.cpp

int deltaX(); - вычисляет смещение перевернутой области по оси абсцисс.

int deltaY(); - вычисляет смещение перевернутой области по оси ординат.

void rotate(); - поворачивает выделенную область изображения на 90/180/270 градусов, используя заданные значения верхнего левого и нижнего правого углов области и угла поворота.

4.7 flagsProcessing.cpp

std::unordered_map<std::string, std::string> getArgs(); - производит разбор командной строки с помощью *getopt()* и записывает результат в хеш-таблицу парами *<аргумент, значение>* для быстрого и удобного доступа к данным.

void isCorrectFlags(); - производит вторичный разбор командной строки, в котором определяет наличие лишних, либо пропущенных команд и, в случае их нахождения, выводит пользователю соответствующее сообщение.

std::string getFuncName(); - определяет среди всех переданных аргументов название вызванной опции, также проверяет ее наличие и единственность.

4.8 validateArgs.cpp

std::vector<std::string> split(); - аналог *strtok()* или *split()* из python. Используется для обработки значений аргументов, заданных через точку. Возвращает массив разделенных значений.

void isValidCoords(); - проверяет корректность формата заданных координат с помощью регулярных строк.

void isValidColor(); - проверяет корректность формата заданного цвета с помощью регулярных строк, также проверяет корректность значений компонент.

bool validateDrawSquare(); - проверяет корректность всех аргументов для вызова функции *–squared_lines*.

void validateRGBfilter(); - проверяет корректность всех аргументов для вызова функции *–rgbfilter*.

void validateRotate(); - проверяет корректность всех аргументов для вызова функции *–rotate*.

bool validateCircle(); - проверяет корректность всех аргументов для вызова функции *–circle*.

void isCorrectArgs(); - на основе выбранной пользователем опции запускает необходимую проверку.

4.9 wrapper.cpp

void fullCheck(); - на основе выбранной пользователем опции передает хеш-таблицу, полученную *getArgs()*, на все необходимые проверки, такие как *isCorrectFlags()* и *isCorrectArgs()*.

void launchFunc(); - производит запуск выбранной опции.

4.10 readBmp.cpp

BMPImage readBmp(); - производит чтение bmp-файла по заданному пути и возвращает структуру *BMPImage* со всей считанной информацией.

4.11 writeBmp.cpp

void writeBmp(); - производит сохранение обработанного изображения по указанному пути.

4.12 printInfo.cpp

void printInfo(); - печатает подробную информацию об изображении.

void printHelp(); - печатает подробную справку о программе.

void useHelp(); - печатает шаблон использования программы и инструкцию по вызову полной справки. Всегда печатается при возникновении ошибок.

5. ПОЛУЧЕННЫЕ РЕЗУЛЬТАТЫ

Программа демонстрирует успешное и корректное выполнение поставленных задач, а также обладает следующими преимуществами:

- Эффективные алгоритмы рисования - использование эффективных алгоритмов рисования, призванных увеличить скорость работы программы.
- Архитектура – проект разбит на несколько частей, каждая из которых выполняет свою задачу, четко структурирован, процесс выполнения задач поделен на этапы, что обеспечивает удобство работы и читаемость при работе с проектом.
- Качество выполнения задач – все алгоритмы выполнения поставленных пользователем задач четко отточены и выполняются корректно.
- Метод сборки – проект собран с помощью *Makefile*, что облегчает редактирование кода и процесс компиляции.
- Обработка ошибок – в программе предусмотрены всевозможные сценарии ошибок, при возникновении которых программа завершает работу с выводом подробной и понятной пользователю информацией.
- Подробная справка-инструкция о программе – в проекте предусмотрен вывод подробной инструкции к использованию программы с описанием всех опций, тонкостей и особенностей работы, призванной помочь любому пользователю быстро разобраться в функционале. Инструкция по вызову данной справки печатается при любой ошибке, чтобы не понимающий в чем дело пользователь мог лучше разобраться в работе программы.

6. ТЕСТИРОВАНИЕ

Пример работы –*circle*:

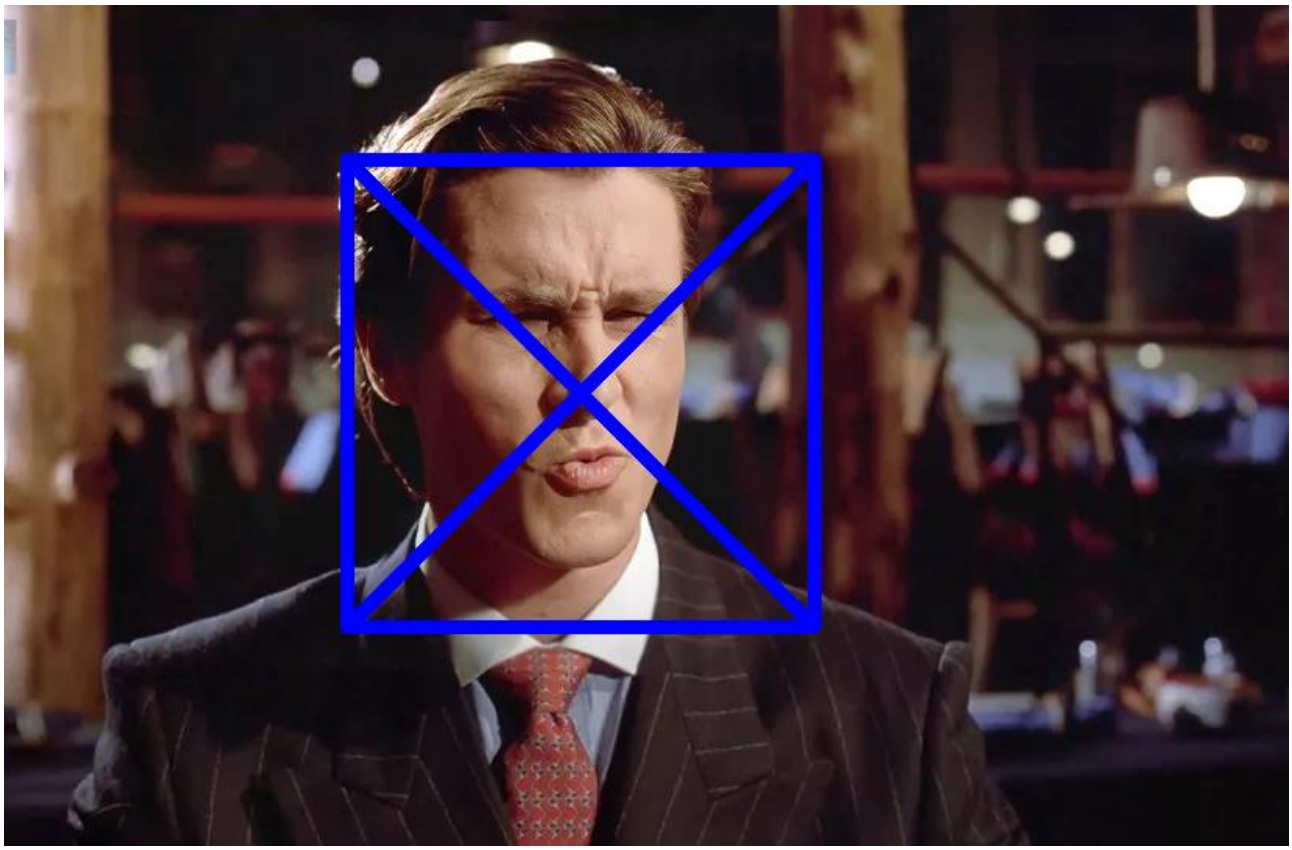
```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --circle --center 350.250
--thickness 10 --radius 100 --color 222.0.222 -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --circle --center 3
50.250 --thickness 10 --radius 100 --color 222.0.222 --fill --fill_color 0.255.0 -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --circle --radiu
s 300 --center 0.0 --thickness 9 --color 99.255.4  -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

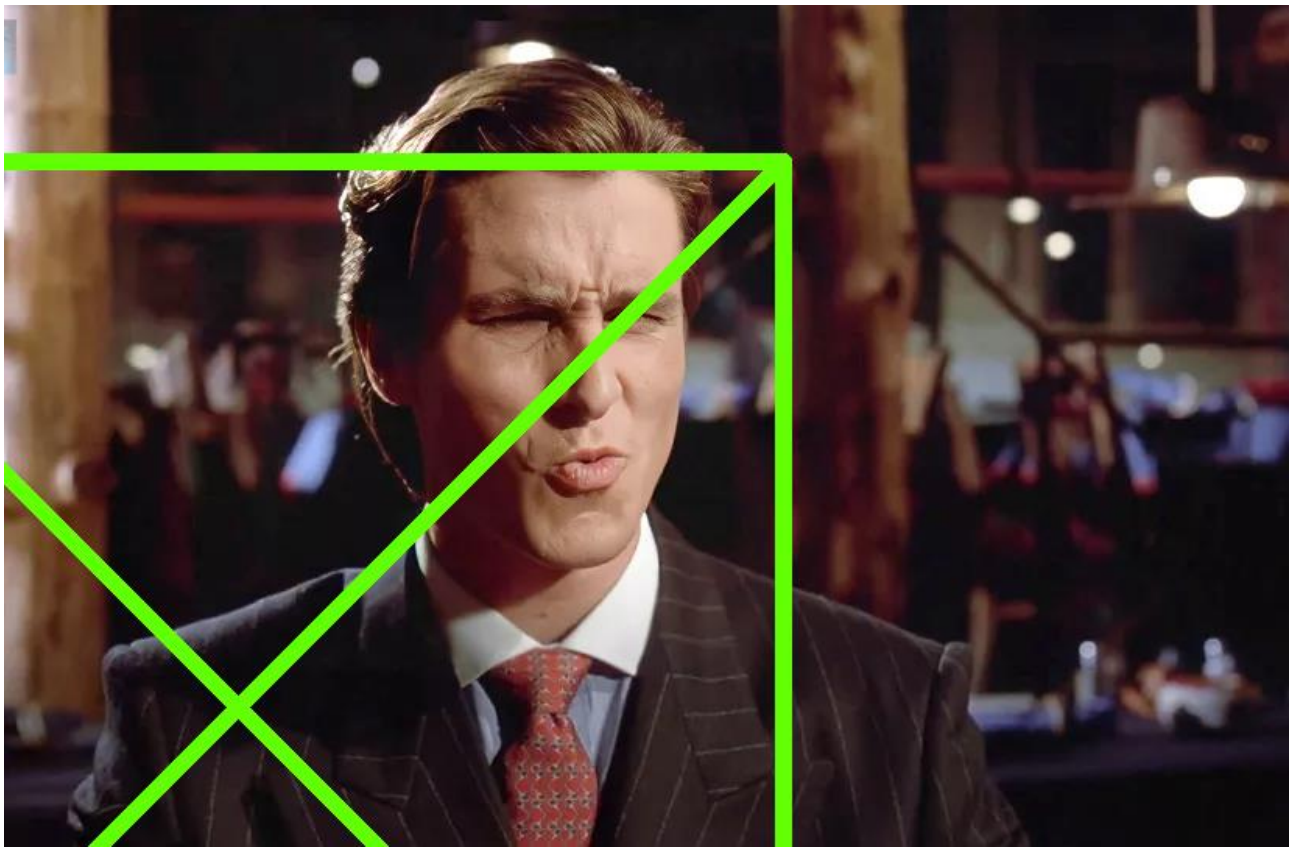
Пример работы –*squared_lines*:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines --left_up
220.99 --side_size 300 --thickness 7 --color 0.0.255 -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```



```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines --left_
up 220.99 --side_size 300 --thickness 7 --color 0.0.255 --fill --fill_color 0.255.255 -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines --left_
up -200.100 --side_size 700 --thickness 10 --color 0.255.0 -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```



Пример работы *–rgbfilter*:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rgbfilter --component_na
me green --component_value 0 -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Пример работы *–rotate*:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rotate --left_up 200.100
 --right_down 500.400 --angle 180 -i TEST2.bmp
```

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rotate --left_up 200.100
 --right_down 500.400 --angle 270 -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rotate --left_
up 0.0 --right_down 1000.1000 --angle 90 -i TEST2.bmp
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Пример работы *–info*:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --info -i TEST2.bmp
___BMP_File_Header_Info___
signature:      19778 (4d42)
filesize:       1337178 (14675a)
reserved1:      0 (0)
reserved2:      0 (0)
pixelArrOffset: 138(8a)

___BMP_Info_Header_Info___
headerSize:     124 (7c)
width:  825 (339)
height: 540 (21c)
planes: 1 (1)
bitsPerPixel:   24 (18)
compression:    0 (0)
imageSize:      1337040 (1466d0)
xPixelsPerMeter:        0 (0)
yPixelsPerMeter:        0 (0)
importantColorCount:    0 (0)
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Пример работы *–help*:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw -h
Course work for option 5.9, created by Mikhail Murdasov.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Обработка ИЗОБРАЖЕНИЙ.
Пример: ./cw --rgbfilter --component_name red --component_value 255 --input img.bmp --outpu
t out.bmp

Аргументы, обязательные для длинных параметров, обязательны и для коротких.
Рисование квадрата с диагоналями:
  --squared_lines              выполнение операции
  --left_up=[ЧИС.ЧИС]          координаты верхнего левого угла
  --side_size=[ЧИС]            размер стороны
  --thickness=[ЧИС]            толщина линий
  --color=[ЧИС.ЧИС.ЧИС]        цвет линий
  --fill                       залить квадрат
  --fill_color=[ЧИС.ЧИС.ЧИС]   цвет заливки

Фильтр RGB-компонент:
  --rgbfilter                  выполнение операции
  --component_name=[ЦВЕТ]      изменяемая компонента
  --component_value=[ЧИС]      новое значение компоненты

Поворот изображения (части) на 90/180/270 градусов:
  --rotate                     выполнение операции
  --left_up=[ЧИС.ЧИС]          координаты верхнего левого угла области
  --right_down=[ЧИС.ЧИС]       координаты верхнего левого угла области
  --angle=[ЧИС]                угол поворота

Рисование окружности:
  --circle                     выполнение операции
  --center=[ЧИС.ЧИС]           координаты центра окружности
  --radius=[ЧИС]               радиус окружности
  --thickness=[ЧИС]            толщина линии окружности
  --color=[ЧИС.ЧИС.ЧИС]        цвет линии
  --fill                       залить окружность
  --fill_color=[ЧИС.ЧИС.ЧИС]   цвет заливки

Прочее:
  -i, --input=[ПУТЬ]           путь к входному изображению
  -o, --output=[ПУТЬ]          путь к выходному изображению
  -h, --help                   вывести подробное описание
  --info                       вывести подробную информацию об изображении
```

```
ФОРМАТЫ:
Значение координат       (--left_up/--right_down/--center): [X.Y], где X - координата по x,
 Y - координата по y.
Значения имени компонент (--component_name):                [ЦВЕТ], где ЦВЕТ принимает знач
ения "red"/"green"/"blue".
Значения компонент       (--component_value):               [ЧИС], где ЧИС - число не меньш
ее 0 и не большее 255.
Значения цвета           (--color/--fill_color):            [RED.GREEN.BLUE], где RED, GREE
N, BLUE - числа не меньшие 0 и не большие 255.
Значение размера стороны (--side_size):                     [ЧИС] - где ЧИС - число, больше
е 0.
Значение толщины линий   (--thickness):                     [ЧИС] - где ЧИС - число, больше
е 0.
Значение угла поворота   (--angle):                         [ЧИС] - где ЧИС принимает значе
ния 90/180/270.
Значение радиуса         (--radius):                        [ЧИС] - где ЧИС - число, больше
е 0.

Коды выхода:
 0 - все отлично,
 40 - ошибка чтения файла,
 41 - ошибка записи файла,
 42 - некорректный файл,
 43 - некорректный параметр,
 44 - некорректный аргумент,
 45 - некорректное завершение работы.

ПРИМЕЧАНИЕ:
Утилита работает только с файлами формата BMP, БЕЗ СЖАТИЯ, имеющими 24 БИТА на цвет.
Параметр --input НЕ является обязательным. При его отсутствии в качестве пути к входному фа
йлу будет взят последний заданный аргумент.
Если ВЫХОДНОЙ ФАЙЛ не указан, то результат по умолчанию сохраняется в текущую директорию с
именем "out.bmp".
Началом оси координат является верхний левый угол. Ось X расположена по горизонтали и растё
т вправо. Ось Y расположена по вертикали и растёт вниз.
Параметры --fill и --fill_color НЕ являются обязательными, но НЕ могут быть использованы др
уг без друга.
Все вводимые числовые значения не должны выходить из диапазона от -2 147 483 648 до 2 147 4
83 647.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

7. ПРИМЕРЫ ОБРАБОТКИ ОШИБОК

Вызов без аргументов:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw
cw: Пропущены аргументы.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Вызов без имени операции:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw -i TEST2.bmp
cw: Не указана операция для выполнения.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Вызов двух и более операций:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --info --circle -i TEST2.bmp
cw: Используйте одну операцию.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Неизвестный параметр:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --moevm
cw: Нераспознанный параметр «--moevm»

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Отсутствие входного файла:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --info
cw: Не указан входной файл.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Неизвестные аргументы:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rgbfilter --component_name red
green blue --component_value 1 2 3 -i test.bmp
cw: Неизвестные аргументы: green, blue, 2, 3

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Совпадение имен входного и выходного файлов:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rgbfilter --component_name blue
 --component_value 1 --input test.bmp --output test.bmp
cw: Входной и выходной файлы должны иметь разные имена.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Использование параметров заливки:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --circle --center 0.0 --radius 10
 --thickness 100 --color 0.0.0 --fill -i TEST2.bmp
cw: Пропущены обязательные параметры: <--fill_color>

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --circle --center 0.0 --radius 10
 --thickness 100 --color 0.0.0 --fill_color 111.111.111 -i TEST2.bmp
cw: Вы не используете <--fill>. Параметр <--fill_color> будет проигнорирован.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Пропуск обязательных параметров:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines --side_size 90 -i
 test.bmp
cw: Пропущены обязательные параметры: <--left_up> <--thickness> <--color>

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Ввод лишних параметров:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --info --angle 90 -i TEST2.bmp
cw: Параметр <--angle> не используется выбранной опцией.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Несуществующий или поврежденный файл:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --info -i 123.bmp
cw: Невозможно открыть файл: 123.bmp

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Некорректные координаты:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rotate --left_up 10l.ahaha --r
ght_down 10.10 --angle 90 -i 123.bmp
cw: Некорректный формат параметра <--left_up>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rotate --left_up -100.10 --rig
t_down ahahaha.ahahaha.ahahah --angle 90 -i 123.bmp
cw: Некорректный формат параметра <--right_down>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --circle --center -сто.сто --rad
us 1 --thickness 1 --color 0.0.0 -i 123.bmp
cw: Некорректный формат параметра <--center>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Некорректно заданный цвет:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --circle --center 100.100 --radi
s 1 --thickness 1 --color 0.0.haha0 -i 123.bmp
cw: Некорректный формат параметра <--color>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --circle --center 100.100 --radi
s 1 --thickness 1 --color 0.0.0 --fill --fill_color -100.-134.moevm -i 123.bmp
cw: Некорректный формат параметра <--fill_color>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Некорректно заданная компонента цвета:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines
--left_up -200.100 --side_size 700 --thickness 10 --color 999.255.0 -i TEST2.bmp
cw: Некорректное значение RED в параметре <--color>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines
--left_up -200.100 --side_size 700 --thickness 10 --color 0.999.0 -i TEST2.bmp
cw: Некорректное значение GREEN в параметре <--color>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines
--left_up -200.100 --side_size 700 --thickness 10 --color 0.0.999 -i TEST2.bmp
cw: Некорректное значение BLUE в параметре <--color>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Разделение ошибок на ошибки «формата» и «значения» для большего понимания пользователя:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines --left_up 0.0 --s
ide_size 0 --thickness -100 --color 0.0.0 -i 123.bmp
cw: Некорректное значение параметра <--sise_size>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --squared_lines --left_up 0.0 --s
ide_size aaaa --thickness -100 --color 0.0.0 -i 123.bmp
cw: Некорректный формат параметра <--side_size>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Некорректное расположение координат относительно друг друга:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rotate --left_up 1000.1000 --ri
ght_down 10.10 --angle 90 -i 123.bmp
cw: Возможно, вы перепутали <--left_up> и <--right_down>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

Несоответствующее типу *int* значение параметра:

```
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$ ./cw --rgbfilter -
-component_name red --component_value 99999999999999999999 -i TEST2.bmp
cw: Возможно, вы ввели слишком большое/маленькое значение параметра <--component_value>.

Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…
Запустите «./cw --help» для получения более подробного описания.
mikhail_murdasov@chesluchilos:~/Рабочий стол/Murdasov_Mikhail_cw/src$
```

ИНСТРУКЦИЯ ПО ЗАПУСКУ

Для запуска приложения необходимо выполнить следующий перечень действий:

1. Откройте директорию, в которой находится проект, через терминал.
2. Введите команду *make* в консоль
3. Введите команду по данному шаблону: *./cw [опция]… [параметры]… [входной файл]… [выходной файл]…*
4. Для вывода справки введите *./cw –help*.
5. По завершении работы с программой введите в консоль команду *make clean*.

ЗАКЛЮЧЕНИЕ

Программа была успешно реализована. Были соблюдены все исходные данные задания. Результаты работы программы соответствуют ожидаемым. Явных ошибок в работе проекта выявлено не было. В процессе работы над проектом были получены навыки работы со стандартной библиотекой C++, файлами формата bmp и интерфейсом командной строки. Цель проекта достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Базовые сведения к выполнению курсового проекта по дисциплине «Программирование». Второй семестр: учеб.-метод. Пособие сост. М. М. Заславский А. А. Лисс А. В. Гаврилов С. А. Глазунов и др. СПб.: Изд-во СПбГЭТУ «ЛЭТИ», 2024.

2. Описание стандартной библиотеки языка C++ // cplusplus.com. URL: https://cplusplus.com/reference/ (Дата обращения 10.04.2024)

3. BMP file format – Википедия // ru.wikipedia.org. URL: https://ru.wikipedia.org/wiki/BMP#BITMAPFILEHEADER (Дата обращения 10.04.2024)

4. MAN getopt // opennet.ru. URL: https://www.opennet.ru/man.shtml?topic=getopt&category=3&russian=0 (Дата обращения 10.04.2024)

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#include "CLI_functions/wrapper.hpp"
#include "CLI_functions/readBmp.hpp"
#include "CLI_functions/writeBmp.hpp"

int
main(int argc, char** argv) {

    if (argc < 2) {
        std::cout << "cw: Пропущено название опции" << std::endl;
        useHelp();
        return 43;
    }

    try {
        std::unordered_map<std::string,    std::string>    argsMap    =
getArgs(argc, argv);

        std::string funcToRun = getFuncName(argsMap);

        fullCheck(funcToRun, argsMap);

        BMPImage bmp = readBmp(argsMap["input"]);

        launchFunc(bmp, funcToRun, argsMap);

        if (argsMap.find("output") != argsMap.end()) {
            writeBmp(bmp, argsMap["output"]);
        }
    }
    catch (std::exception &e) {
        std::cout << e.what() << std::endl;
        std::cout << "cw: Некорректное завершение работы\n";
        return 45;
    }

    return 0;
}
```

Название файла: flagsProcessing.cpp

```
#include "flagsProcessing.hpp"

struct option
long_options[] = {
    {"input", required_argument, nullptr, 'i'},
    {"output", required_argument, nullptr, 'o'},
    {"squared_lines", no_argument, nullptr, 0},
    {"left_up", required_argument, nullptr, 0},
    {"side_size", required_argument, nullptr, 0},
    {"thickness", required_argument, nullptr, 0},
    {"color", required_argument, nullptr, 0},
    {"fill", no_argument, nullptr, 0},
    {"fill_color", required_argument, nullptr, 0},
    {"rgbfilter", no_argument, nullptr, 0},
```

```
    {"component_name", required_argument, nullptr, 0},
    {"component_value", required_argument, nullptr, 0},
    {"rotate", no_argument, nullptr, 0},
    {"right_down", required_argument, nullptr, 0},
    {"angle", required_argument, nullptr, 0},
    {"circle", no_argument, nullptr, 0},
    {"center", required_argument, nullptr, 0},
    {"radius", required_argument, nullptr, 0},
    {"info", no_argument, nullptr, 0},
    {"help", no_argument, nullptr, 'h'},
    {0, 0, 0, 0},

};

std::unordered_map<std::string, std::string>
getArgs(int argc, char** argv) {

    std::unordered_map<std::string, std::string> argsMap;
    std::string option_name;
    int option = 0;
    opterr = 0;
    int ind = 0;

    argsMap["output"] = "out.bmp";

    bool user_output = false;

     while ((option = getopt_long(argc, argv, "i:o:h", long_options,
&ind)) != -1) {

        if (option == '?') {
            std::cerr << "cw: Нераспознанный параметр «" << argv[optind
- 1] << "»" << std::endl;
            useHelp();
            exit(43);
        }

        if (option == 'i') {
            option_name = "input";
        } else if (option == 'o') {
            user_output = true;
            argsMap.erase("output");
            option_name = "output";
        } else if (option == 'h') {
            printHelp();
            exit(0);
        } else {
            option_name = long_options[ind].name;
        }

        if (optarg) {
            argsMap.insert(std::make_pair(option_name, optarg));
        } else {
            argsMap.insert(std::make_pair(option_name, ""));
        }

    }
```

```
    if (argsMap.find("input") == argsMap.end()) {
        if (optind < argc) {
            argsMap.insert(std::make_pair("input", argv[argc - 1]));
            optind++;
        } else {
            std::cerr << "cw: Не указан входной файл." << std::endl;
            useHelp();
            exit(40);
        }
    }

    if (optind < argc) {
        std::cerr << "cw: Неизвестные аргументы: ";
        for (int i = optind; i < argc; i++) {
            std::cerr << argv[i] << ", ";
        }
        std::cerr << "\b\b \b\b" << std::endl;
        useHelp();
        exit(44);
    }

    if (getFuncName(argsMap) == "--info" && user_output == false) {
        argsMap.erase("output");
    }

    if (argsMap.find("output") != argsMap.end() && argsMap["input"] ==
argsMap["output"]) {
        std::cerr << "cw: Входной и выходной файлы должны иметь разные
имена." << std::endl;
        useHelp();
        exit(44);
    }

    return argsMap;

}

void
isCorrectFlags(std::vector<std::string>                          flags,
std::unordered_map<std::string, std::string> argsMap) {

    std::vector<std::string> extraFlags;
    std::vector<std::string> missingFlags;

    bool correct = true;

    for (const auto& key : flags) {
        if (argsMap.find(key) == argsMap.end()) {
            if (key != "fill" && key != "fill_color"){
                correct = false;
                missingFlags.push_back(key);
            }
        }
    }

            if     (argsMap.find("fill")     ==     argsMap.end()     &&
argsMap.find("fill_color") != argsMap.end() &&
```

```
          find(flags.begin(), flags.end(), "fill") != flags.end() &&
find(flags.begin(), flags.end(), "fill_color") != flags.end()) {
        //correct = false;
        //missingFlags.push_back("fill");
          std::cerr << "cw: Вы не используете <--fill>. Параметр <--
fill_color> будет проигнорирован." << std::endl;
        }   else   if   (argsMap.find("fill")   !=   argsMap.end()   &&
argsMap.find("fill_color") == argsMap.end() &&
          find(flags.begin(), flags.end(), "fill") != flags.end() &&
find(flags.begin(), flags.end(), "fill_color") != flags.end()) {
        correct = false;
        missingFlags.push_back("fill_color");
    }

    for (const auto& [key, value] : argsMap) {
        if (find(flags.begin(), flags.end(), key) == flags.end()) {
            correct = false;
            extraFlags.push_back(key);
        }
    }

    if (correct == false) {

        if (extraFlags.empty() == false) {
            for (const auto& item: extraFlags) {
                   std::cerr << "cw: Параметр <--" << item << "> не
используется выбранной опцией." << std::endl;
            }
        }

        if (missingFlags.empty() == false) {
            std::cout << "cw: Пропущены обязательные параметры: ";
            for (const auto& item : missingFlags) {
                std::cout << "<--" << item << "> ";
            }
            std::cout << std::endl;

        }

        useHelp();
        exit(43);
    }

}

std::string
getFuncName(std::unordered_map<std::string, std::string> argsMap) {
    std::string funcName;
    int cnt = 0;
    for (const auto& [key, val] : argsMap) {
        if (key == "squared_lines") {
            funcName = key;
            cnt++;
        } else if (key == "rgbfilter") {
            funcName = key;
            cnt++;
        } else if (key == "rotate") {
            funcName = key;
```

```
            cnt++;
        } else if (key == "circle") {
            funcName = key;
            cnt++;
        } else if (key == "info") {
            funcName = key;
            cnt++;
        }
    }

    if (cnt == 0) {
          std::cerr << "cw: Не указана операция для выполнения." <<
std::endl;
        useHelp();
        exit(43);
    } else if (cnt > 1) {
        std::cerr << "cw: Используйте одну операцию." << std::endl;
        useHelp();
        exit(43);
    } else if (cnt == 1) {
        return "--" + funcName;
    }

}
```

Название файла: validateArgs.cpp

```
#include "validateArgs.hpp"

void
isCorrectArgs(std::string   funcName,   std::unordered_map<std::string,
std::string> argsMap) {

    if (funcName == "--squared_lines") {
        std::string left_up, side_size, thickness, color, fill_color;

        bool fill_check = false;

        for (const auto& [key, val] : argsMap) {
            if (key == "left_up") left_up = val;
            if (key == "side_size") side_size = val;
            if (key == "thickness") thickness = val;
            if (key == "color") color = val;
            if (key == "fill") fill_check = true;
            if (key == "fill_color") fill_color = val;
        }

        if (fill_check == false) {
            validateDrawSquare(left_up, side_size, thickness, color);
        } else {
              validateDrawSquare(left_up, side_size, thickness, color,
fill_color);
        }
    }

    if (funcName == "--rgbfilter") {
        std::string component_name, component_value;

        for (const auto& [key, val] : argsMap) {
```

```
            if (key == "component_name") component_name = val;
            if (key == "component_value") component_value = val;
        }

        validateRGBfilter(component_name, component_value);
    }

    if (funcName == "--circle") {
        std::string center, radius, thickness, color, fill_color;

        bool fill_check = false;

        for (const auto& [key, val] : argsMap) {
            if (key == "center") center = val;
            if (key == "radius") radius = val;
            if (key == "thickness") thickness = val;
            if (key == "color") color = val;
            if (key == "fill") fill_check = true;
            if (key == "fill_color") fill_color = val;
        }

        if (fill_check == false) {
            validateCircle(center, radius, thickness, color);
        } else {
           validateCircle(center, radius, thickness, color, fill_color);
        }
    }

    if (funcName == "--rotate") {
        std::string left_up, right_down, angle;

        for (const auto& [key, val] : argsMap) {
            if (key == "left_up") left_up = val;
            if (key == "right_down") right_down = val;
            if (key == "angle") angle = val;
        }

        validateRotate(left_up, right_down, angle);
    }
}


std::vector<std::string>
split(const std::string& str, const std::string& delim) {
    std::vector<std::string> tokens;
    size_t start = 0;
    size_t end = 0;
    while ((end = str.find(delim, start)) != std::string::npos) {
        tokens.push_back(str.substr(start, end - start));
        start = end + delim.length();
    }
    tokens.push_back(str.substr(start));
    return tokens;
}

void
isValidCoords(std::string coords, std::string arg) {
    std::regex pattern_coord("^-?\\d+\\.-?\\d+$");
```

```
    try {
        if(std::regex_match(coords, pattern_coord) == false){
            throw std::runtime_error("cw: Некорректный формат параметра
<" + arg + ">.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }
    std::vector<std::string> coords_val = split(coords, ".");
    try {
        int x = stoi(coords_val[0]);
        int y = stoi(coords_val[1]);
    } catch (std::out_of_range& err) {
        std::cout << "cw: Возможно, вы ввели слишком большое/маленькое
значение параметра <" + arg + ">." << std::endl;
        useHelp();
        exit(44);
    }

}

void
isValidColor(std::string color, std::string arg) {
    std::regex pattern_color("^\\d+\\.\\d+\\.\\d+$");
    try {
        if(std::regex_match(color, pattern_color) == false) {
            throw std::runtime_error("cw: Некорректный формат параметра
<" + arg + ">.");
        } else {

            int color_r, color_g, color_b;
            std::vector<std::string> colors_arr = split(color, ".");

            try {
                color_r = stoi(colors_arr[0]);
                color_g = stoi(colors_arr[1]);
                color_b = stoi(colors_arr[2]);
            }
            catch (std::out_of_range& err) {
                      std::cout << "cw: Возможно, вы ввели слишком
большое/маленькое значение параметра <" + arg + ">." << std::endl;
                useHelp();
                exit(44);
            }

                  if (!(color_r >= 0 && color_r <= 255)) throw
std::runtime_error("cw: Некорректное значение RED в параметре <" + arg
+ ">.");
                  if (!(color_g >= 0 && color_g <= 255)) throw
std::runtime_error("cw: Некорректное значение GREEN в параметре <" + arg
+ ">.");
                  if (!(color_b >= 0 && color_b <= 255)) throw
std::runtime_error("cw: Некорректное значение BLUE в параметре <" + arg
+ ">.");
```

```
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }
}

bool
validateDrawSquare(std::string    left_up,    std::string    side_size,
std::string thickness, std::string color) {

    std::regex pattern_size("^\\d+$");
    std::regex pattern_thickness("^\\d+$");

    isValidCoords(left_up, "--left_up");

    try {
        if (std::regex_match(side_size, pattern_size) == false) {
            throw std::runtime_error("cw: Некорректный формат параметра
<--side_size>.");
        } else {

            int size;
            try {
                size = stoi(side_size);
            }
            catch (std::out_of_range& err) {
                      std::cout  <<  "cw:  Возможно,  вы  ввели  слишком
большое/маленькое значение параметра <--side_size>." << std::endl;
                useHelp();
                exit(44);
            }
             if (size <= 0) throw std::runtime_error("cw: Некорректное
значение параметра <--sise_size>.");

        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    try {
        if (std::regex_match(thickness, pattern_thickness) == false) {
            throw std::runtime_error("cw: Некорректный формат параметра
<--thickness>.");
        } else {

            int thickness_int;
            try {
                thickness_int = stoi(thickness);
            }
            catch (std::out_of_range& err) {
                      std::cout  <<  "cw:  Возможно,  вы  ввели  слишком
большое/маленькое значение параметра <--thickness>." << std::endl;
```

```
                useHelp();
                exit(44);
            }

               if (thickness_int <= 0) throw std::runtime_error("cw:
Некорректное значение параметра <--thickness>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    isValidColor(color, "--color");

    return true;

}

bool
validateDrawSquare(std::string left_up, std::string side_size,
std::string thickness, std::string color, std::string fill_color) {

    std::regex pattern_size("^\\d+$");
    std::regex pattern_thickness("^\\d+$");

    isValidCoords(left_up, "--left_up");

    try {
        if (std::regex_match(side_size, pattern_size) == false) {
            throw std::runtime_error("cw: Некорректный формат параметра
<--side_size>.");
        } else {

            int size;
            try {
                size = stoi(side_size);
            }
            catch (std::out_of_range& err) {
                      std::cout << "cw: Возможно, вы ввели слишком
большое/маленькое значение параметра <--side_size>." << std::endl;
                useHelp();
                exit(44);
            }
             if (size <= 0) throw std::runtime_error("cw: Некорректное
значение параметра <--sise_size>.");

        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    try {
        if (std::regex_match(thickness, pattern_thickness) == false) {
```

```
            throw std::runtime_error("cw: Некорректный формат параметра
<--thickness>.");
        } else {

            int thickness_int;
            try {
                thickness_int = stoi(thickness);
            }
            catch (std::out_of_range& err) {
                      std::cout << "cw: Возможно, вы ввели слишком
большое/маленькое значение параметра <--thickness>." << std::endl;
                useHelp();
                exit(44);
            }

               if (thickness_int <= 0) throw std::runtime_error("cw:
Некорректное значение параметра <--thickness>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    isValidColor(color, "--color");

    isValidColor(fill_color, "--fill_color");

    return true;

}

void
validateRGBfilter(std::string        component_name,        std::string
component_value) {

    std::regex pattern_value("^\\d+$");

    try {
         if (!(component_name == "red" || component_name == "green" ||
component_name == "blue")) {
           throw std::runtime_error("cw: Некорректное значение параметра
<--component_name>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    try {
         if (std::regex_match(component_value, pattern_value) == false)
{
            throw std::runtime_error("cw: Некорректный формат параметра
<--component_value>.");
        } else {
```

```
            int value;
            try {
                value = stoi(component_value);
            }
            catch (std::out_of_range& err) {
                      std::cout << "cw: Возможно, вы ввели слишком
большое/маленькое значение параметра <--component_value>." << std::endl;
                useHelp();
                exit(44);
            }

                    if (!(value >= 0 && value <= 255)) throw
std::runtime_error("cw: Некорректное значение параметра <--
component_value>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);

    }

}

void
validateRotate(std::string left_up, std::string right_down, std::string
angle) {

    isValidCoords(left_up, "--left_up");

    isValidCoords(right_down, "--right_down");

    try {
        if (!(angle == "90" || angle == "180" || angle == "270")) {
           throw std::runtime_error("cw: Некорректное значение параметра
<--angle>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    std::vector<std::string> coords_up = split(left_up, ".");
    int x_up = stoi(coords_up[0]);
    int y_up = stoi(coords_up[1]);

    std::vector<std::string> coords_down = split(right_down, ".");
    int x_down = stoi(coords_down[0]);
    int y_down = stoi(coords_down[1]);

    try {
        if (!(x_up <= x_down && y_up <= y_down)) { //
             throw std::runtime_error("cw: Возможно, вы перепутали <--
left_up> и <--right_down>.");
```

```
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

}

bool
validateCircle(std::string  center,  std::string  radius,  std::string
thickness, std::string color) {

    std::regex pattern_radius("^\\d+$");
    std::regex pattern_thickness("^\\d+$");

    isValidCoords(center, "--center");

    try {
        if (std::regex_match(radius, pattern_radius) == false) {
            throw std::runtime_error("cw: Некорректный формат параметра
<--radius>.");
        } else {

            int value;
            try {
                value = stoi(radius);
            }
            catch (std::out_of_range& err) {
                      std::cout  <<  "cw:  Возможно,  вы  ввели  слишком
большое/маленькое значение параметра <--radius>." << std::endl;
                useHelp();
                exit(44);
            }

             if (value <= 0) throw std::runtime_error("cw: Некорректное
значение параметра <--radius>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    try {
        if (std::regex_match(thickness, pattern_thickness) == false) {
            throw std::runtime_error("cw: Некорректный формат параметра
<--thickness>.");
        } else {

            int thickness_int;
            try {
                thickness_int = stoi(thickness);
            }
            catch (std::out_of_range& err) {
```

```
                    std::cout << "cw: Возможно, вы ввели слишком
большое/маленькое значение параметра <--thickness>." << std::endl;
                useHelp();
                exit(44);
            }

                if (thickness_int <= 0) throw std::runtime_error("cw:
Некорректное значение параметра <--thickness>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    isValidColor(color, "--color");

    return true;
}

bool
validateCircle(std::string center, std::string radius, std::string
thickness, std::string color, std::string fill_color) {

    std::regex pattern_radius("^\\d+$");
    std::regex pattern_thickness("^\\d+$");

    isValidCoords(center, "--center");

    try {
        if (std::regex_match(radius, pattern_radius) == false) {
            throw std::runtime_error("cw: Некорректный формат параметра
<--radius>.");
        } else {

            int value;
            try {
                value = stoi(radius);
            }
            catch (std::out_of_range& err) {
                    std::cout << "cw: Возможно, вы ввели слишком
большое/маленькое значение параметра <--radius>." << std::endl;
                useHelp();
                exit(44);
            }

             if (value <= 0) throw std::runtime_error("cw: Некорректное
значение параметра <--radius>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    try {
```

```
        if (std::regex_match(thickness, pattern_thickness) == false) {
            throw std::runtime_error("cw: Некорректный формат параметра
<--thickness>.");
        } else {

            int thickness_int;
            try {
                thickness_int = stoi(thickness);
            }
            catch (std::out_of_range& err) {
                      std::cout << "cw: Возможно, вы ввели слишком
большое/маленькое значение параметра <--thickness>." << std::endl;
                useHelp();
                exit(44);
            }

               if (thickness_int <= 0) throw std::runtime_error("cw:
Некорректное значение параметра <--thickness>.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(44);
    }

    isValidColor(color, "--color");

    isValidColor(fill_color, "--fill_color");

    return true;
}
```

Название файла: wrapper.cpp

```
#include "wrapper.hpp"

void
fullCheck(std::string    firstFlag,    std::unordered_map<std::string,
std::string> argsMap) {

    std::vector<std::string> squaredFlags = {"squared_lines", "left_up",
"side_size", "thickness", "color", "fill", "fill_color", "input",
"output"};
        std::vector<std::string>   rgbFlags          =   {"rgbfilter",
"component_name", "component_value", "input", "output"};
      std::vector<std::string>  rotateFlags   =  {"rotate",  "left_up",
"right_down", "angle", "input", "output"};
      std::vector<std::string>  circleFlags   =  {"circle",  "center",
"radius", "thickness", "color", "fill", "fill_color", "input", "output"};
    std::vector<std::string> infoFlags    = {"info", "input"};

    if (firstFlag == "--squared_lines") {
        isCorrectFlags(squaredFlags, argsMap);
        isCorrectArgs("--squared_lines", argsMap);

    }
```

```
    if (firstFlag == "--rgbfilter") {
        isCorrectFlags(rgbFlags, argsMap);
        isCorrectArgs("--rgbfilter", argsMap);
    }

    if (firstFlag == "--circle") {
        isCorrectFlags(circleFlags, argsMap);
        isCorrectArgs("--circle", argsMap);
    }

    if (firstFlag == "--rotate") {
        isCorrectFlags(rotateFlags, argsMap);
        isCorrectArgs("--rotate", argsMap);
    }

    if (firstFlag == "--info") {
        isCorrectFlags(infoFlags, argsMap);
    }
}

void
launchFunc(BMPImage&        image,        std::string        funcName,
std::unordered_map<std::string, std::string> argsMap) {

    bool launch = false;

    if (funcName == "--squared_lines") {
        std::string left_up, side_size, thickness, color, fill_color;
        RGBTriple color_push, fill_color_push;

        bool fill = false;

        for (const auto& [key, val] : argsMap) {
            if (key == "left_up") left_up = val;
            if (key == "side_size") side_size = val;
            if (key == "thickness") thickness = val;
            if (key == "color") color = val;
            if (key == "fill") fill = true;
            if (key == "fill_color") fill_color = val;
        }

        std::vector<std::string> coords = split(left_up, ".");
        int x = stoi(coords[0]);
        int y = stoi(coords[1]);

        int side_size_push = stoi(side_size);
        int thickness_push = stoi(thickness);

        std::vector<std::string> color_arr = split(color, ".");
        color_push.rgbRed = stoul(color_arr[0]);
        color_push.rgbGreen = stoul(color_arr[1]);
        color_push.rgbBlue = stoul(color_arr[2]);

        if (fill == true) {
            std::vector<std::string> fill_color_arr = split(fill_color,
".");
            fill_color_push.rgbRed = stoul(fill_color_arr[0]);
```

```
            fill_color_push.rgbGreen = stoul(fill_color_arr[1]);
            fill_color_push.rgbBlue = stoul(fill_color_arr[2]);

              draw_square(image, x, y, side_size_push, thickness_push,
color_push, true, fill_color_push);
            launch = true;
        } else {
              draw_square(image, x, y, side_size_push, thickness_push,
color_push, false, {0,0,0});
            launch = true;
        }

    }

    if (funcName == "--rgbfilter") {
        std::string component_name, component_value;

        for (const auto& [key, val] : argsMap) {
            if (key == "component_name") component_name = val;
            if (key == "component_value") component_value = val;
        }

        int component_value_push = stoi(component_value);

        rgbFilter(image, component_name, component_value_push);
        launch = true;
    }

    if (funcName == "--circle") {
        std::string center, radius, thickness, color, fill_color;
        RGBTriple color_push, fill_color_push;

        bool fill = false;

        for (const auto& [key, val] : argsMap) {
            if (key == "center") center = val;
            if (key == "radius") radius = val;
            if (key == "thickness") thickness = val;
            if (key == "color") color = val;
            if (key == "fill") fill = true;
            if (key == "fill_color") fill_color = val;
        }

        std::vector<std::string> coords = split(center, ".");
        int center_x = stoi(coords[0]);
        int center_y = stoi(coords[1]);

        int radius_push = stoi(radius);
        int thickness_push = stoi(thickness);

        std::vector<std::string> color_arr = split(color, ".");
        color_push.rgbRed = stoul(color_arr[0]);
        color_push.rgbGreen = stoul(color_arr[1]);
        color_push.rgbBlue = stoul(color_arr[2]);

        if (fill == true) {
            std::vector<std::string> fill_color_arr = split(fill_color,
".");
```

```
            fill_color_push.rgbRed = stoul(fill_color_arr[0]);
            fill_color_push.rgbGreen = stoul(fill_color_arr[1]);
            fill_color_push.rgbBlue = stoul(fill_color_arr[2]);

                  drawCircle(image,  center_x,  center_y,  radius_push,
thickness_push, color_push, true, fill_color_push);
            launch = true;
        } else {
                  drawCircle(image,  center_x,  center_y,  radius_push,
thickness_push, color_push, false, {0, 0, 0});
            launch = true;
        }

    }

    if (funcName == "--rotate") {
        std::string left_up, right_down, angle;

        for (const auto& [key, val] : argsMap) {
            if (key == "left_up") left_up = val;
            if (key == "right_down") right_down = val;
            if (key == "angle") angle = val;
        }

        std::vector<std::string> coords_up = split(left_up, ".");
        int x_up = stoi(coords_up[0]);
        int y_up = stoi(coords_up[1]);

        std::vector<std::string> coords_down = split(right_down, ".");
        int x_down = stoi(coords_down[0]);
        int y_down = stoi(coords_down[1]);

        int angle_push = stoi(angle);

        rotate(image, x_up, y_up, x_down, y_down, angle_push);
        launch = true;

    }

    if (funcName == "--info") {
        printInfo(image);
        launch = true;
    }

    if (!launch) {
        std::cerr << "cw: Несуществующая операция <" + funcName + ">."
<< std::endl;
        useHelp();
        exit(43);
    }

}
```

Название файла: readBmp.cpp

```
#include "readBmp.hpp"

BMPImage
readBmp(const std::string& filename) {
```

```
    std::ifstream file(filename, std::ios::binary);

    try {
        if (!file.is_open()) {
            throw std::runtime_error("cw: Невозможно открыть файл: " +
filename);
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(40);
    }

    BMPImage image;
    file.read((char*)&image.fileHeader, sizeof(BMPFileHeader));

    try {
        if (image.fileHeader.bfSignature != 0x4D42) {
            throw std::runtime_error("cw: Данный файл не является файлом
формата BMP.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(42);
    }

    file.read((char*)&image.infoHeader, sizeof(BMPInfoHeader));

    try {
        if (image.infoHeader.biBitPerPixel != 24) {
            throw std::runtime_error("cw: Данный файл не имеет 24 бита
на цвет.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(42);
    }

    try {
        if (image.infoHeader.biCompression != 0) {
            throw   std::runtime_error("cw:   Данный   файл   использует
сжатие.");
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(42);
    }
```

```
    file.seekg(image.fileHeader.bfFileOffsetToPixelArray,
std::ios::beg);

    unsigned    int    padding    =    (image.infoHeader.biWidth    *
sizeof(RGBTriple)) % 4;
    if (padding) padding = 4 - padding;

    std::vector<std::vector<RGBTriple>>
    pixels_arr(abs(image.infoHeader.biHeight),
std::vector<RGBTriple>(image.infoHeader.biWidth));

    for (int i = image.infoHeader.biHeight - 1; i >= 0; i--) {
        for (int j = 0; j < image.infoHeader.biWidth; j++) {
            RGBTriple pixel;
            file.read((char*)&pixel, sizeof(RGBTriple));
            pixels_arr[i][j] = pixel;
        }
        file.seekg(padding, std::ios::cur);
    }

    image.pixelsArr.resize(abs(image.infoHeader.biHeight),
std::vector<RGBTriple>(image.infoHeader.biWidth));
    image.pixelsArr = pixels_arr;

    file.close();

    return image;
}
```

Название файла: writeBmp.cpp

```
#include "writeBmp.hpp"

void
writeBmp(const BMPImage& image, const std::string& filename) {

    std::ofstream file(filename, std::ios::binary);

    try {
        if (!file.is_open()) {
            throw std::runtime_error("cw: Невозможно открыть файл: " +
filename);
        }
    }
    catch (std::runtime_error& e) {
        std::cout << e.what() << std::endl;
        useHelp();
        exit(41);
    }

    file.write((char*)&image.fileHeader, sizeof(BMPFileHeader));

    file.write((char*)&image.infoHeader, sizeof(BMPInfoHeader));

    file.seekp(image.fileHeader.bfFileOffsetToPixelArray,
std::ios::beg);

    unsigned    int    padding    =    (image.infoHeader.biWidth    *
sizeof(RGBTriple)) % 4;
```

```
    if (padding) padding = 4 - padding;

    for (int i = image.infoHeader.biHeight - 1; i >= 0; i--) {

        file.write((char*)image.pixelsArr[i].data(),
(image.pixelsArr[i].size())*sizeof(RGBTriple));
        for (int j = 0; j < padding; j++) {
            char zero = 0;
            file.write((char*)&zero, 1);
        }
    }


    file.close();


}
```

Название файла: drawCircle.cpp

```
#include "drawCircle.hpp"

void
fillCircle(BMPImage& image, int center_x, int center_y, int r, RGBTriple
color) {

    if (r == 1) {
        if (center_y + 1 >= 0 && center_y + 1 < image.infoHeader.biHeight
&&
            center_x >= 0 && center_x < image.infoHeader.biWidth) {

            image.pixelsArr[center_y + 1][center_x] = color;
        }
        if (center_y - 1 >= 0 && center_y - 1 < image.infoHeader.biHeight
&&
            center_x >= 0 && center_x < image.infoHeader.biWidth) {

            image.pixelsArr[center_y - 1][center_x] = color;
        }
        if (center_y >= 0 && center_y < image.infoHeader.biHeight &&
            center_x - 1 >= 0 && center_x - 1 < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y][center_x - 1] = color;
        }
        if (center_y >= 0 && center_y < image.infoHeader.biHeight &&
            center_x + 1 >= 0 && center_x + 1 < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y][center_x + 1] = color;
        }
        if (center_y >= 0 && center_y < image.infoHeader.biHeight &&
            center_x >= 0 && center_x < image.infoHeader.biWidth) {

            image.pixelsArr[center_y][center_x] = color;
        }

        return;
    }
```

```
    int x = 0;
    int y = r;
    int delta = 3 - 2 * y;

    while (y >= x) {
        if (center_y + y >= 0 && center_y + y < image.infoHeader.biHeight
&&
            center_x + x >= 0 && center_x + x < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y + y][center_x + x] = color;
        }
        if (center_y - y >= 0 && center_y - y < image.infoHeader.biHeight
&&
            center_x + x >= 0 && center_x + x < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y - y][center_x + x] = color;
        }
        if (center_y + y >= 0 && center_y + y < image.infoHeader.biHeight
&&
            center_x - x >= 0 && center_x - x < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y + y][center_x - x] = color;
        }
        if (center_y - y >= 0 && center_y - y < image.infoHeader.biHeight
&&
            center_x - x >= 0 && center_x - x < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y - y][center_x - x] = color;
        }
        if (center_y + x >= 0 && center_y + x < image.infoHeader.biHeight
&&
            center_x + y >= 0 && center_x + y < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y + x][center_x + y] = color;
        }
        if (center_y - x >= 0 && center_y - x < image.infoHeader.biHeight
&&
            center_x + y >= 0 && center_x + y < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y - x][center_x + y] = color;
        }
        if (center_y + x >= 0 && center_y + x < image.infoHeader.biHeight
&&
            center_x - y >= 0 && center_x - y < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y + x][center_x - y] = color;
        }
        if (center_y - x >= 0 && center_y - x < image.infoHeader.biHeight
&&
```

```
            center_x - y >= 0 && center_x - y < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y - x][center_x - y] = color;
        }

        delta += delta < 0 ? 4 * x + 6 : 4 * (x - y--) + 10;
        ++x;
    }

    for (int y = -r; y <= r; y++) {
        if      ((center_y      +      y)<0      ||      (center_y      +
y)>=image.infoHeader.biHeight) {
            continue;
        }
        for (int x = -r; x <= r; x++) {
            if                  (((center_x+x)>=0)                  &&
((center_x+x)<image.infoHeader.biWidth) && (x * x + y * y <= r * r)) {
                image.pixelsArr[center_y + y][center_x + x] = color;
            }
        }
    }
}


void
drawCircle(BMPImage& image, int center_x, int center_y, int r, int
thickness, RGBTriple color, bool fill, RGBTriple fill_color) {

    int x = 0;
    int radius;
    int y = r;
    int delta = 1 - 2 * r;
    int error = 0;

    if (thickness % 2 == 0) {
        radius = thickness / 2;
    } else if (thickness == 1) {
        radius = 0;
    } else {
        radius = (thickness + 1) / 2;
    }

    if (fill == true) {
        fillCircle(image, center_x, center_y, r, fill_color);
    }

    if (r == 1) {
        if (center_y + 1 >= 0 && center_y + 1 < image.infoHeader.biHeight
&&
            center_x >= 0 && center_x < image.infoHeader.biWidth) {

            image.pixelsArr[center_y + 1][center_x] = color;
        }
        if (center_y - 1 >= 0 && center_y - 1 < image.infoHeader.biHeight
&&
            center_x >= 0 && center_x < image.infoHeader.biWidth) {
```

```
            image.pixelsArr[center_y - 1][center_x] = color;
        }
        if (center_y >= 0 && center_y < image.infoHeader.biHeight &&
            center_x - 1 >= 0 && center_x - 1 < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y][center_x - 1] = color;
        }
        if (center_y >= 0 && center_y < image.infoHeader.biHeight &&
            center_x + 1 >= 0 && center_x + 1 < image.infoHeader.biWidth)
{

            image.pixelsArr[center_y][center_x + 1] = color;
        }

        return;
    }

    while (y >= x) {

        fillCircle(image, center_x + x, center_y + y, radius, color);
        fillCircle(image, center_x + x, center_y - y, radius, color);
        fillCircle(image, center_x - x, center_y + y, radius, color);
        fillCircle(image, center_x - x, center_y - y, radius, color);
        fillCircle(image, center_x + y, center_y + x, radius, color);
        fillCircle(image, center_x + y, center_y - x, radius, color);
        fillCircle(image, center_x - y, center_y + x, radius, color);
        fillCircle(image, center_x - y, center_y - x, radius, color);

        error = 2 * (delta + y) - 1;
        if ((delta < 0) && (error <= 0)) {
            delta += 2 * ++x + 1;
            continue;
        }
        if ((delta > 0) && (error > 0)) {
            delta -= 2 * --y + 1;
            continue;
        }

        delta += 2 * (++x - --y);
    }
}
```

Название файла: drawSquare.cpp

```
#include "drawSquare.hpp"
#include "drawCircle.hpp"

void
draw_square(BMPImage &image, int x, int y, int side_size, int thickness,
RGBTriple color, bool fill, RGBTriple fill_color) {

    if (fill == true) {
        for (int i = y + 1; i < y + side_size; i++) {
            for (int j = x + 1; j < x + side_size; j++) {
                if (i < 0 || j < 0 || i >= image.infoHeader.biHeight ||
j >= image.infoHeader.biWidth) {
                    continue;
                }
```

```
                image.pixelsArr[i][j] = fill_color;
            }
        }
    }

    if (thickness == 1) {
        for (int i = y; i < y + side_size; i++) {
            if (i >= 0 && i < image.infoHeader.biHeight &&
                x >= 0 && x < image.infoHeader.biWidth) {

                image.pixelsArr[i][x] = color;
            }

            if (i >= 0 && i < image.infoHeader.biHeight &&
                        x + side_size >= 0 && x + side_size <
image.infoHeader.biWidth) {

                image.pixelsArr[i][x + side_size] = color;
            }
        }

        for (int i = x; i < x + side_size; i++) {
            if (y >= 0 && y < image.infoHeader.biHeight &&
                i >= 0 && i < image.infoHeader.biWidth) {

                image.pixelsArr[y][i] = color;
            }

            if (i >= 0 && i < image.infoHeader.biWidth &&
                        y + side_size > 0 && y + side_size <
image.infoHeader.biHeight) {

                image.pixelsArr[y + side_size][i] = color;
            }
        }

        for (int i = y; i < y + side_size; i++) {
            for (int j = x; j < x + side_size; j++) {
                if (i - y == j - x) {

                    if (i >= 0 && i < image.infoHeader.biHeight &&
                        j >= 0 && j < image.infoHeader.biWidth) {
                            image.pixelsArr[i][j] = color;
                        }

                    if (i >= 0 && i < image.infoHeader.biHeight &&
                        side_size + 2 * x - j < image.infoHeader.biWidth
&& side_size + 2 * x - j >= 0) {
                            image.pixelsArr[i][side_size + 2 * x - j] =
color;
                    }

                }
            }
        }

        return;
    }
```

```
    int radius = 0;
    if (thickness % 2 == 0) {
        radius = thickness / 2;
    } else if (thickness == 1) {
        radius = 0;
    } else {
        radius = (thickness + 1) / 2;
    }

    for (int i = x; i < x + side_size; i++) {
        if (i < 0 || i >= image.infoHeader.biWidth) {
            continue;
        }
        fillCircle(image, i, y, radius, color);
        fillCircle(image, i, y + side_size, radius, color);
    }

    for (int i = y; i < y + side_size; i++) {
        if (i < 0 || i >= image.infoHeader.biHeight) {
            continue;
        }
        fillCircle(image, x, i, radius, color);
        fillCircle(image, x + side_size, i, radius, color);
    }

    for (int i = y; i < y + side_size; i++) {
        for (int j = x; j < x + side_size; j++) {
            if (i - y == j - x) {
                fillCircle(image, j, i, radius, color);
                  fillCircle(image, side_size + 2 * x - j, i, radius,
color);
            }
        }
    }
}

void draw_line(BMPImage& image, int x, int y, int x1, int y1, int
thickness, RGBTriple color){
    int delta_x = abs(x1 - x);
    int delta_y = abs(y1 - y);
    int sign_x = x < x1 ? 1 : -1;
    int sign_y = y < y1 ? 1 : -1;
    int error = delta_x - delta_y, error2;
    /*int radius;
    if (thickness % 2 == 0) {
        radius = thickness / 2;
    } else if (thickness == 1) {
        radius = 0;
    } else {
        radius = (thickness + 1) / 2;
    }*/

    while (x != x1 || y != y1) {
        if (y >= 0 && y < image.infoHeader.biHeight &&
            x >= 0 && x < image.infoHeader.biWidth) {
            if (thickness == 1) {
                image.pixelsArr[y][x] = color;
```

```
            } else {
                fillCircle(image, x, y, thickness/2 , color);
            }

            error2 = 2 * error;
            if (error2 > -delta_y) {
                error -= delta_y;
                x += sign_x;
            }
            if (error2 < delta_x) {
                error += delta_x;
                y += sign_y;
            }
        }
    }
}
```

Название файла: rgbFilter.cpp

```
#include "rgbFilter.hpp"

void
rgbFilter(BMPImage&    image,    std::string    component_name,    int
component_value) {

    if (component_name == "red") {
        for (int y = 0; y < image.infoHeader.biHeight; y++) {
            for (int x = 0; x < image.infoHeader.biWidth; x++) {
                image.pixelsArr[y][x].rgbRed = component_value;
            }
        }
    } else if (component_name == "green") {
        for (int y = 0; y < image.infoHeader.biHeight; y++) {
            for (int x = 0; x < image.infoHeader.biWidth; x++) {
                image.pixelsArr[y][x].rgbGreen = component_value;
            }
        }
    } else if (component_name == "blue") {
        for (int y = 0; y < image.infoHeader.biHeight; y++) {
            for (int x = 0; x < image.infoHeader.biWidth; x++) {
                image.pixelsArr[y][x].rgbBlue = component_value;
            }
        }
    }
}
```

Название файла: rotate.cpp

```
#include "rotate.hpp"

int
deltaX (int angle, int x, int y) {

    if (angle == 90) {
        if (x%2 == 0 && y%2 == 0) return 0;
        if (x%2 != 0 && y%2 != 0) return 0;
        if (x%2 == 0 && y%2 != 0) return 0;
        if (x%2 != 0 && y%2 == 0) return 0; //
    }
```

```
    if (angle == 180) {
        if (x%2 != 0 && y%2 == 0) return 0;
        if (x%2 != 0 && y%2 != 0) return 0;
        if (x%2 == 0 && y%2 != 0) return -1;
        if (x%2 == 0 && y%2 == 0) return -1;
    }

    if (angle == 270) {
        if (x%2 != 0 && y%2 != 0) return 0;
        if (x%2 != 0 && y%2 == 0) return -1;
        if (x%2 == 0 && y%2 == 0) return -1;
        if (x%2 == 0 && y%2 != 0) return 0; //
    }

    return 0;

}

int
deltaY (int angle, int x, int y) {

    if (angle == 90) {
        if (x%2 == 0 && y%2 == 0) return -1;
        if (x%2 != 0 && y%2 != 0) return 0;
        if (x%2 == 0 && y%2 != 0) return -1;
        if (x%2 != 0 && y%2 == 0) return 0; //
    }

    if (angle == 180) {
        if (x%2 != 0 && y%2 == 0) return -1;
        if (x%2 != 0 && y%2 != 0) return 0;
        if (x%2 == 0 && y%2 != 0) return 0;
        if (x%2 == 0 && y%2 == 0) return -1;
    }

    if (angle == 270) {
        if (x%2 != 0 && y%2 != 0) return 0;
        if (x%2 != 0 && y%2 == 0) return 0;
        if (x%2 == 0 && y%2 == 0) return 0;
        if (x%2 == 0 && y%2 != 0) return 0; //
    }

    return 0;

}

void
rotate(BMPImage &image, int x, int y, int x1, int y1, int angle) {

    if (angle % 360 == 0) {
        return;
    }

    std::vector<std::vector<RGBTriple>> new_elem = image.pixelsArr;

    double radians = (angle * M_PI/180);
```

```
    if (x < 0) {
        x = 0;
    } else if (x >= image.infoHeader.biWidth) {
        x = image.infoHeader.biWidth;
    }

    if (y < 0) {
        y = 0;
    } else if (y >= image.infoHeader.biHeight) {
        y = image.infoHeader.biHeight;
    }

    if (x1 < 0) {
        x1 = 0;
    } else if (x1 >= image.infoHeader.biWidth) {
        x1 = image.infoHeader.biWidth;
    }

    if (y1 < 0) {
        y1 = 0;
    } else if (y1 >= image.infoHeader.biHeight) {
        y1 = image.infoHeader.biHeight;
    }

    int cx = (x + x1) / 2;
    int cy = (y + y1) / 2;

    if (angle == 180) {
        for (int i = y; i < y1; i++) {
            for (int j = x; j < x1; j++) {
                if (i < 0 || j < 0 || i >= image.infoHeader.biHeight ||
j >= image.infoHeader.biWidth) {
                    continue;
                }

                int dx = deltaX(angle, x1 - x, y1 - y);
                int dy = deltaY(angle, x1 - x, y1 - y);

                double i_old = (cy - i);
                double j_old = (cx - j);

                if (cy + round(i_old) + dy >= 0 && cy + round(i_old) +
dy < image.infoHeader.biHeight &&
                    cx + round(j_old) + dx >= 0 && cx + round(j_old) +
dx < image.infoHeader.biWidth) {
                    image.pixelsArr[i][j] = new_elem[cy + round(i_old)
+ dy][cx + round(j_old) + dx];
                }
            }
        }

        return;
    }

    for (int i = y; i < y1; i++) {
        for (int j = x; j < x1; j++) {
            if (i < 0 || j < 0 || i >= image.infoHeader.biHeight || j >=
image.infoHeader.biWidth) {
```

```
                continue;
            }

            int dx = deltaX(angle, x1 - x, y1 - y);
            int dy = deltaY(angle, x1 - x, y1 - y);

            double i_old = (cy - i);
            double j_old = (cx - j);

            double i_new = j_old * sin(radians) + i_old * cos(radians)
+ dy;
            double j_new = j_old * cos(radians) - i_old * sin(radians)
+ dx;

            if (cy + round(i_new) >= 0 && cy + round(i_new) <
image.infoHeader.biHeight &&
                cx + round(j_new) >= 0 && cx + round(j_new) <
image.infoHeader.biWidth) {
                image.pixelsArr[round(i_new) + cy][round(j_new) + cx] =
new_elem[i][j];
            }
        }
    }

}
```

Название файла: printInfo.cpp

```
#include "printInfo.hpp"

void
printInfo(BMPImage& image) {

    std::cout << "___BMP_File_Header_Info___" << std::endl;
            std::cout << "signature:\t" << std::dec <<
image.fileHeader.bfSignature << " (" << std::hex <<
image.fileHeader.bfSignature << ")" << std::endl;
            std::cout << "filesize:\t" << std::dec <<
image.fileHeader.bfFileSize << " (" << std::hex <<
image.fileHeader.bfFileSize << ")" << std::endl;
            std::cout << "reserved1:\t" << std::dec <<
image.fileHeader.bfReserved1 << " (" << std::hex <<
image.fileHeader.bfReserved1 << ")" << std::endl;
            std::cout << "reserved2:\t" << std::dec <<
image.fileHeader.bfReserved2 << " (" << std::hex <<
image.fileHeader.bfReserved2 << ")" << std::endl;
          std::cout << "pixelArrOffset:\t" << std::dec <<
image.fileHeader.bfFileOffsetToPixelArray << "(" << std::hex <<
image.fileHeader.bfFileOffsetToPixelArray << ")" << std::endl;

    std::cout << "\n___BMP_Info_Header_Info___" << std::endl;
    std::cout << "headerSize:\t" << std::dec << image.infoHeader.biSize
<< " (" << std::hex << image.infoHeader.biSize << ")" << std::endl;
    std::cout << "width:\t" << std::dec << image.infoHeader.biWidth <<
" (" << std::hex << image.infoHeader.biWidth << ")" << std::endl;
    std::cout << "height:\t" << std::dec << image.infoHeader.biHeight
<< " (" << std::hex << image.infoHeader.biHeight << ")" << std::endl;
    std::cout << "planes:\t" << std::dec << image.infoHeader.biPlanes
<< " (" << std::hex << image.infoHeader.biPlanes << ")" << std::endl;
```

```
          std::cout    <<    "bitsPerPixel:\t"    <<    std::dec    <<
image.infoHeader.biBitPerPixel    <<    "    ("    <<    std::hex    <<
image.infoHeader.biBitPerPixel << ")" << std::endl;
          std::cout    <<    "compression:\t"    <<    std::dec    <<
image.infoHeader.biCompression    <<    "    ("    <<    std::hex    <<
image.infoHeader.biCompression << ")" << std::endl;
           std::cout    <<    "imageSize:\t"    <<    std::dec    <<
image.infoHeader.biImageSize    <<    "    ("    <<    std::hex    <<
image.infoHeader.biImageSize << ")" << std::endl;
         std::cout    <<    "xPixelsPerMeter:\t"    <<    std::dec    <<
image.infoHeader.biXPixelsPerMeter    <<    "    ("    <<    std::hex    <<
image.infoHeader.biXPixelsPerMeter << ")" << std::endl;
         std::cout    <<    "yPixelsPerMeter:\t"    <<    std::dec    <<
image.infoHeader.biYPixelsPerMeter    <<    "    ("    <<    std::hex    <<
image.infoHeader.biYPixelsPerMeter << ")" << std::endl;
        std::cout    <<    "importantColorCount:\t"    <<    std::dec    <<
image.infoHeader.biImportantColors    <<    "    ("    <<    std::hex    <<
image.infoHeader.biImportantColors << ")" << std::endl;

}

void
printHelp() {

     std::cout << "Course work for option 5.9, created by Mikhail
Murdasov.\n" << std::endl;

     std::cout << "Использование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ
ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…" << std::endl;

    std::cout << "Обработка ИЗОБРАЖЕНИЙ." << std::endl;

     std::cout << "Пример: ./cw --rgbfilter --component_name red --
component_value 255 --input img.bmp --output out.bmp" << std::endl;

     std::cout << "\nАргументы, обязательные для длинных параметров,
обязательны и для коротких." << std::endl;
    std::cout << "Рисование квадрата с диагоналями:" << std::endl;
    std::cout << "  --squared_lines                выполнение операции"
<< std::endl;
    std::cout << "  --left_up=[ЧИС.ЧИС]            координаты верхнего
левого угла" << std::endl;
    std::cout << "  --side_size=[ЧИС]              размер стороны" <<
std::endl;
    std::cout << "  --thickness=[ЧИС]              толщина линий" <<
std::endl;
    std::cout << "  --color=[ЧИС.ЧИС.ЧИС]        цвет линий" << std::endl;
    std::cout << "  --fill                         залить квадрат" <<
std::endl;
    std::cout << "  --fill_color=[ЧИС.ЧИС.ЧИС]    цвет заливки\n" <<
std::endl;

    std::cout << "Фильтр RGB-компонент:" << std::endl;
    std::cout << "  --rgbfilter                    выполнение операции"
<< std::endl;
    std::cout << "  --component_name=[ЦВЕТ]      изменяемая компонента"
<< std::endl;
```

```
      std::cout << "   --component_value=[ЧИС]         новое значение
компоненты\n" << std::endl;

    std::cout << "Поворот изображения (части) на 90/180/270 градусов:"
<< std::endl;
    std::cout << "  --rotate                        выполнение операции"
<< std::endl;
     std::cout << "  --left_up=[ЧИС.ЧИС]            координаты верхнего
левого угла области" << std::endl;
     std::cout << "  --right_down=[ЧИС.ЧИС]         координаты верхнего
левого угла области" << std::endl;
    std::cout << "  --angle=[ЧИС]                   угол поворота\n" <<
std::endl;

    std::cout << "Рисование окружности:" << std::endl;
    std::cout << "  --circle                        выполнение операции"
<< std::endl;
     std::cout << "  --center=[ЧИС.ЧИС]              координаты центра
окружности" << std::endl;
    std::cout << "  --radius=[ЧИС]                 радиус окружности" <<
std::endl;
   std::cout << "  --thickness=[ЧИС]            толщина линии окружности"
<< std::endl;
   std::cout << "  --color=[ЧИС.ЧИС.ЧИС]        цвет линии" << std::endl;
    std::cout << "  --fill                          залить окружность" <<
std::endl;
     std::cout << "  --fill_color=[ЧИС.ЧИС.ЧИС]    цвет заливки\n" <<
std::endl;

    std::cout << "Прочее:" << std::endl;
     std::cout << "  -i, --input=[ПУТЬ]              путь к входному
изображению" << std::endl;
     std::cout << "  -o, --output=[ПУТЬ]             путь к выходному
изображению" << std::endl;
     std::cout << "  -h, --help                      вывести подробное
описание" << std::endl;
     std::cout << "  --info                          вывести подробную
информацию об изображении\n" << std::endl;

    std::cout << "ФОРМАТЫ:" << std::endl;
     std::cout << "Значение координат        (--left_up/--right_down/--
center): [X.Y], где X - координата по x, Y - координата по y." <<
std::endl;
           std::cout    <<    "Значения    имени    компонент    (--
component_name):                  [ЦВЕТ], где ЦВЕТ принимает значения
\"red\"/\"green\"/\"blue\"." << std::endl;
          std::cout    <<    "Значения    компонент                 (--
component_value):                [ЧИС], где ЧИС - число не меньшее 0 и
не большее 255." << std::endl;
       std::cout   <<   "Значения   цвета                    (--color/--
fill_color):            [RED.GREEN.BLUE], где RED, GREEN, BLUE - числа
не меньшие 0 и не большие 255." << std::endl;
           std::cout    <<    "Значение    размера    стороны    (--
side_size):                      [ЧИС] - где ЧИС - число, большее 0." <<
std::endl;
          std::cout    <<    "Значение    толщины    линий         (--
thickness):                      [ЧИС] - где ЧИС - число, большее 0." <<
std::endl;
```

```
          std::cout    <<    "Значение    угла    поворота         (--
angle):                            [ЧИС] - где ЧИС принимает значения
90/180/270." << std::endl;
         std::cout    <<    "Значение    радиуса                    (--
radius):                         [ЧИС] - где ЧИС - число, большее 0.\n"
<< std::endl;

    std::cout << "Коды выхода:" << std::endl;
    std::cout << " 0 - все отлично," << std::endl;
    std::cout << " 40 - ошибка чтения файла," << std::endl;
    std::cout << " 41 - ошибка записи файла," << std::endl;
    std::cout << " 42 - некорректный файл," << std::endl;
    std::cout << " 43 - некорректный параметр," << std::endl;
    std::cout << " 44 - некорректный аргумент," << std::endl;
    std::cout << " 45 - некорректное завершение работы.\n" << std::endl;

    std::cout << "ПРИМЕЧАНИЕ:" << std::endl;
     std::cout << "Утилита работает только с файлами формата BMP, БЕЗ
СЖАТИЯ, имеющими 24 БИТА на цвет." << std::endl;
     std::cout << "Параметр --input НЕ является обязательным. При его
отсутствии в качестве пути к входному файлу будет взят последний заданный
аргумент." << std::endl;
    std::cout << "Если ВЫХОДНОЙ ФАЙЛ не указан, то результат по умолчанию
сохраняется в текущую директорию с именем \"out.bmp\"." << std::endl;
    std::cout << "Началом оси координат является верхний левый угол. Ось
X расположена по горизонтали и растёт вправо. Ось Y расположена по
вертикали и растёт вниз." << std::endl;
      std::cout  <<  "Параметры  --fill  и  --fill_color  НЕ  являются
обязательными, но НЕ могут быть использованы друг без друга." <<
std::endl;
    std::cout << "Все вводимые числовые значения не должны выходить из
диапазона от −2 147 483 648 до 2 147 483 647." << std::endl;

}

void useHelp() {
    std::cout << "\nИспользование: ./cw [ОПЦИЯ]… [ПАРАМЕТРЫ]… [ВХОДНОЙ
ФАЙЛ]… [ВЫХОДНОЙ ФАЙЛ]…\nЗапустите «./cw --help» для получения более
подробного описания." << std::endl;
}
```

Название файла: flagsProcessing.hpp

```
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>
#include <getopt.h>
#include <algorithm>

#include "../bmp_structs.hpp"
#include "validateArgs.hpp"
#include "../image_functions/drawSquare.hpp"
#include "../printInfo.hpp"

std::string  getFuncName(std::unordered_map<std::string,  std::string>
argsMap);
```

```
std::unordered_map<std::string, std::string> getArgs(int argc, char**
argv);

void isCorrectFlags(std::vector<std::string> flags,
std::unordered_map<std::string, std::string> argsMap);
```

Название файла: readBmp.hpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include "../bmp_structs.hpp"
#include "../printInfo.hpp"

BMPImage readBmp(const std::string& filename);
```

Название файла: validateArgs.hpp

```
#include <iostream>
#include <regex>
#include <unordered_map>
#include <string>
#include <vector>

#include "../printInfo.hpp"
#include "../bmp_structs.hpp"

std::vector<std::string> split(const std::string& str, const
std::string& delim);

void isValidCoords(std::string coords, std::string arg);

void isValidColor(std::string color, std::string arg);

bool validateDrawSquare(std::string left_up, std::string side_size,
std::string thickness, std::string color);

bool validateDrawSquare(std::string left_up, std::string side_size,
std::string thickness, std::string color, std::string fill_color);

void validateRGBfilter(std::string component_name, std::string
component_value);

void validateRotate(std::string left_up, std::string right_down,
std::string angle);

bool validateCircle(std::string center, std::string radius, std::string
thickness, std::string color);

bool validateCircle(std::string center, std::string radius, std::string
thickness, std::string color, std::string fill_color);

void isCorrectArgs(std::string funcName, std::unordered_map<std::string,
std::string> argsMap);
```

Название файла: wrapper.hpp

```
#include <iostream>
#include <string>
```

```
#include <vector>
#include <unordered_map>
#include <getopt.h>
#include <algorithm>
#include <fstream>
#include <stdexcept>

#include "../bmp_structs.hpp"
#include "flagsProcessing.hpp"
#include "validateArgs.hpp"
#include "../image_functions/drawSquare.hpp"
#include "../image_functions/drawCircle.hpp"
#include "../image_functions/rotate.hpp"
#include "../image_functions/rgbFilter.hpp"
#include "../printInfo.hpp"

void fullCheck(std::string firstFlag, std::unordered_map<std::string,
std::string> argsMap);

void launchFunc(BMPImage& image, std::string funcName,
std::unordered_map<std::string, std::string> argsMap);
```

Название файла: writeBmp.hpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include "../bmp_structs.hpp"
#include "../printInfo.hpp"

void writeBmp(const BMPImage& image, const std::string& filename);
```

Название файла: drawCircle.hpp

```
#include <iostream>
#include <string>
#include <vector>
#include <cmath>

#include "../bmp_structs.hpp"

void drawCircle(BMPImage& image, int center_x, int center_y, int r, int
thickness, RGBTriple color, bool fill, RGBTriple fill_color);

void fillCircle(BMPImage& image, int center_x, int center_y, int r,
RGBTriple color);
```

Название файла: drawSquare.hpp

```
#include "../bmp_structs.hpp"
#include <iostream>

void draw_square(BMPImage& image, int x, int y, int side_size, int
thickness, RGBTriple color, bool fill, RGBTriple fill_color);
```

Название файла: rgbFilter.hpp

```
#include <iostream>
#include <string>
```

```
#include "../bmp_structs.hpp"

void rgbFilter(BMPImage& image, std::string component_name, int
component_value);
```

Название файла: rotate.hpp

```
#include "../bmp_structs.hpp"
#include <cmath>

void rotate(BMPImage &image, int x1, int y1, int x2, int y2, int angle);

int deltaX (int angle, int x, int y);

int deltaY (int angle, int x, int y);
```

Название файла: bmp_structs.hpp

```
#include <vector>
#include <stdint.h>

#pragma once
#pragma pack(push, 1)

typedef struct BMPFileHeader {
    uint16_t bfSignature;
    uint32_t bfFileSize;
    uint16_t bfReserved1;
    uint16_t bfReserved2;
    uint32_t bfFileOffsetToPixelArray;
}BMPFileHeader;

typedef struct BMPInfoHeader {
    uint32_t biSize;
    int32_t biWidth;
    int32_t biHeight;
    uint16_t biPlanes;
    uint16_t biBitPerPixel;
    uint32_t biCompression;
    uint32_t biImageSize;
    int32_t biXPixelsPerMeter;
    int32_t biYPixelsPerMeter;
    uint32_t biTotalColors;
    uint32_t biImportantColors;
}BMPInfoHeader;

typedef struct RGBTriple {
    uint8_t rgbBlue;
    uint8_t rgbGreen;
    uint8_t rgbRed;
} RGBTriple;

typedef struct BMPImage {
    BMPFileHeader fileHeader;
    BMPInfoHeader infoHeader;
    std::vector<std::vector<RGBTriple>> pixelsArr;
}BMPImage;

#pragma pack(pop)
```

Название файла: printInfo.hpp

```
#include <iostream>

#include "bmp_structs.hpp"

void printHelp();
void printInfo(BMPImage& image);
void useHelp();
```

Название файла: Makefile

```
CC = g++
CPPFLAGS = -std=c++11 -O3 -w
LDFLAGS = -lstdc++ -lm
MAKEFLAGS += -j8 --silent
SRCDIRS = . image_functions CLI_functions
SOURCES = $(foreach dir, $(SRCDIRS), $(wildcard $(dir)/*.cpp))
OBJECTS = $(patsubst %.cpp, %.o, $(SOURCES))

%.o: %.cpp
	$(CC) $(CPPFLAGS) -c $< -o $@

cw: $(OBJECTS)
	$(CC) $(LDFLAGS) $(OBJECTS) -o cw

clean:
	rm -f $(OBJECTS) cw
```